



**SZÉCHENYI
EGYETEM**
UNIVERSITY OF GYŐR
GÉPÉSZMÉRNÖKI, INFORMATIKAI
ÉS VILLAMOSMÉRNÖKI KAR

Gépi látás

Paradicsomlevél betegség felismerés

Doma Zsófia Anna

Mérnökinformatikus BSc szak

2026

Tartalom

Bevezetés	2
Projekt célja	3
Felhasznált technológiák.....	3
Adathalmaz bemutatása és műveletek	5
Adatelőkészítés	6
Modell bemutatása	8
Tanítási folyamat.....	9
Kísérletek és eredmények	12
Baseline kísérlet	12
Gaussian noise kísérlet.....	14
Gaussian blur kísérlet.....	16
Irodalomjegyzék	18
Ábrajegyzék	18

Bevezetés

A képfeldolgozás és a gépi látás egyre nagyobb és fontosabb szerepet tölt be napjainkban az élet különböző területein. Gyakran használt technológia az egészségügyben, iparban, biztonságtechnikában és a mezőgazdaságban. A modern fejlesztések lehetővé tették a gépi látás területén, hogy automatikusan fel tudjunk ismerni és osztályozni különböző objektum típusokat és mintázatokat előre elkészített képek alapján. A mezőgazdaságban kifejezetten hasznos technológia ez, hiszen így lehetőség tárul növénybetegségek korai felismerésére, ami befolyásolhatja a terméshozamot és növelheti az élelmiszerbiztonságot.

A növénybetegségek felismerése alapvetően szakemberhez tartozó feladat, aki megfelelő szaktudással rendelkezik. Bizonyos betegségeknek viszont nagyon hasonló vizuális tünetei vannak az adott növényen, így fel lehet ismerni őket a jellegzetes mintákról, színekről. Továbbá nagy mezőgazdasági területeken ahol sok növény van, nehéz és időigényes feladat a manuális ellenőrzés, így a gépi látás használata jövedelmező döntés.

A mély neurális hálózatok, kifejezetten a konvolúciós neurális hálózatok (Convolutional Neural Networks, CNN), jól teljesítenek a képfeldolgozási munkákban. Ezen modellek képesek automatikusan megtanulni az input képeken található fontos jellemzőket, mint például alakzatokat, textúrákat és színmintákat (IBM, 2026).

A projekt célja egy képosztályozó rendszer elkészítése, ami képes paradicsomlevelek különböző betegségeinek automatikus felismerésére. A rendszer megvalósításához PyTorch alapú tanulási modellt alkalmaztam, ami egy előre tanított MobileNetV2 architektúrára épül. A modell a KaggleHub PlantVillage adatkészlet egy részén került betanításra, ami különböző beteg paradicsomleveleket tartalmazó képeket tartalmaz.

A feladat során nem csak a betegségek osztályozása volt a cél, hanem annak elemzése is, hogy különböző zavaró tényezők mellett, hogyan teljesít a modell. Ennek érdekében több különböző kísérlet készült, amelyek során Gaussian blur és Gaussian noise transzformációkat alkalmaztam a képekre. Ezáltal meg lehetett figyelni, hogy a romlott képminőség, milyen hatással van a modellre és mennyire befolyásolja annak teljesítményét.

A később ismertetett eredmények jól szemléltetik, hogy a mélytanulási modellek megfelelő körülmények mellett, képesek magas pontossággal dolgozni növénybetegségek felismerésekor. Viszont bizonyos zavaró komponensek jelentősen csökkenthetik a rendszer megbízhatóságát. A rendszer működése továbbá megmutatja azt is, hogy a bemeneti képek minősége és a megfelelő előkészítésük kardinális szerepet játszanak a teljesítményben.

Projekt célja

A projekt elsődleges célja egy olyan gépi látási modell elkészítése, amely képes paradicsomlevél betegségek osztályozására, felismerésére a megadott képek alapján. A feladat során a mélytanulási modell létrehozása, betanítása és kiértékelése volt a fő vonal. Ezen túl vizsgáltam azt is, hogy megadott képi torzítások mellett, hogyan teljesít a modell.

A PlantVillage adatkészletet használtam a modell tanítására, ezen belül pedig kiválasztottam három betegségtípust, melyek a Tomato Late Blight, Tomato Leaf Mold és Tomato Septorial Leaf Spot, illetve az egészséges leveleket. A modell stabilitásának vizsgálatához három különböző kísérleti formát alkalmaztam. Az első a baseline, ami a modell zavarás nélküli tanítását jelenti. A második esetben Gaussian Blur és a harmadikban Gaussian noise-t alkalmaztam. A baseline célja, hogy legyen egy referencia teljesítmény a tiszta képek alapján. A Gaussian blur az elmosódott képeket modellezte, és a Gaussian noise pedig a rossz képinőségéből származó zajt helyettesítette.

A projektben különböző kiértékelési metrikákat alkalmazta. Nem csak accuracy alapján vizsgáltam, hanem precision, recall és F1-score értékek segítségével is. Továbbá létrehoztam a baseline, blur és noise kísérletekhez egy-egy confusion matrix elemzést is, ami lehetővé tette annak megfigyelését, hogy a modell melyik osztályokat keverte leggyakrabban össze.

A rendszer felépítése egyszerű, átlátható és egyszerűen továbbfejleszthető. Ahoz, hogy ezek megvalósíthatóak legyenek, a program moduláris felépítésű, ahol külön van kezelve az adatelőkészítés, a konfiguráció, az adatkészlet betöltése, a modell tanítása és a tesztelési, kiértékelési folyamatok.

Felhasznált technológiák

A program elkészítése során több gépi tanulási és képfeldolgozási technológia került felhasználásra.

A verziókezeléshez Git-et és GitHub repository volt alkalmazva. Ez lehetővé tette, hogy a fejlesztési folyamat követhető legyen, a módosítások biztonságosan legyenek kezelve, segítette a strukturált tárolást és az egyes verziók visszaállítását. A GitHub repository tartalmazta a forráskódot, a konfigurációs fájlokat valamint a dokumentációt. A nagyméretű futtatási eredmények és adatkészletek .gitignore használatával kizárásra kerültek a repository tartalmából.

A KaggleHub könyvtárban szereplő PlantVillage adatkészletet töltöttem le a modell tanítási

bemenetéhez. A Kaggle adatokat közvetlen el lehet érni Python környezetből. A projekt során így lehetővé tette, hogy automatikusan le tudjam tölteni a szükséges kép adatbázist és reprodukálhatóvá vált a projekt struktúrája (Kaggle, 2018).

Mint már fentebb említettem, a projekt teljes megvalósítása Python programozási nyelven készült. A Python az egyik legnépszerűbb nyelv a mesterséges intelligencia és gépi tanulás területén, hiszen sok könyvtár és eszköz áll rendelkezésre a neurális hálózatok fejlesztéséhez, elemzésre és képfeldolgozásra. A projekt szempontjából számos előnye van a Python használatának, hiszen gyorsan lehet vele prototípust készíteni, széles körű benne a gépi tanulás támogatás és átfogó dokumentációval rendelkezik. A rendszer létrehozása során a Python biztosította, a teljes tanulási folyamat vezérlését, az adatok kezelését, a modell kiértékelését és az eredmények vizualizálását, mentését (Foundation, 2026).

A neurális hálózat megvalósításához PyTorch keretrendszert használtam. A PyTorch egy nyílt forráskódú mélytanulási keretrendszer, melyet a Meta fejlesztett. A projektben biztosította a neurális háló definíálását, a tensor műveleteket, az automatikus differenciálást, a GPU támogatást és a tanítási, tesztelési részfolyamatok megfelelő kezelését. Ezen keretrendszer legnagyobb előnye, hogy dinamikus számítási gráfot használ, ami miatt egyszerűbb a modellek fejlesztése és a hibák keresése. Továbbá jól párosítható egyéb Python könyvtárakkal is. A PyTorch segítségével történt betöltésre a MobilNetV2 is, illetve a loss függvény kezelése, az optimizer használata és a backpropagation végrehajtása is ezzel történt (PyTorch, 2026).

A PyTorch keretrendszeren belül található a torchvision kiegészítő könyvtár, ami tartalmaz több előre elkészített modellt, adatkészlet kezelő egységet és képi transzformációs lehetőségeket. Ennek segítségével került betöltésre az ImageFolder, alkalmazhatóak voltak a képi transzformációk és ezáltal lett megvalósítva az adataugmentáció. A transzformációk között szerepel a képek átméretezése, horizontális tükrözés és a Gaussian blur alkalmazása.

A feladat során különösen fontos volt az ImageFolder osztály használata, hiszen ezáltal automatikusan lehetett képadathalmazokat mappastruktúrába helyezni (PyTorch, 2026).

A rendszerben szereplő neurális hálózat alapjául a MobileNetV2 szolgált. Ez az architektúra egy könnyű, gyors és hatékony konvolúciós neurális hálózat, amit eredetileg mobil eszközökre és beágyazott rendszerekre optimalizáltak. Azért került a MobileNetV2-re a választás mert alacsony a számítási igénye, gyors a tanítási sebessége, jó a képfeldolgozási teljesítménye és elérhetőek benne előre tanított súlyok. A projektben a transfer learning módszer került alkalmazásra, ami azt takarja, hogy a modell már rendelkezett előzetesen megtanult jellemzőkkel az ImageNet adatkészleten lévő tanításból, ezáltal nem volt szükség a nulláról való tanításra. A négy betegséget figyelembevéve, az eredetileg 1000 osztályos kimeneti

réteget lecseréltem négyre (PyTorch, 2026).

Az elkészült modell teljesítményének kiértékeléséhez a scikit-learn könyvtár bizonyos funkcióit alkalmaztam. A scikit-learn egy elterjedte Python aláú géi tanulási könyvtár, ami sok adatfeldolgozási és kiértékelési eszközt tartalmaz. Alkalmaztam a confusion matrix számításához, a classification report generálásához, a precision, recall és F1-score kiszámításához. Ezen értékek lehetővé tették a modell részletes elemzését osztályonként, nem csak accuracy érték alapján (Cournapeau, 2026).

Végezetül, az eredmények vizuális megjelenítéséhez a matplotlib könyvtárat használatam, amely egy Python alapú adatvizualizációs eszköz. Ennek segítségével készült el a confusion matrix ábrája és a grafikus eredmények. Ezáltal jól áttekinthetővé váltak az eredmények és megfigyelhetővé vált, hogy a modell melyik osztályokat keverte össze a különböző kísérletek alatt (Hunter & Dale, 2026).

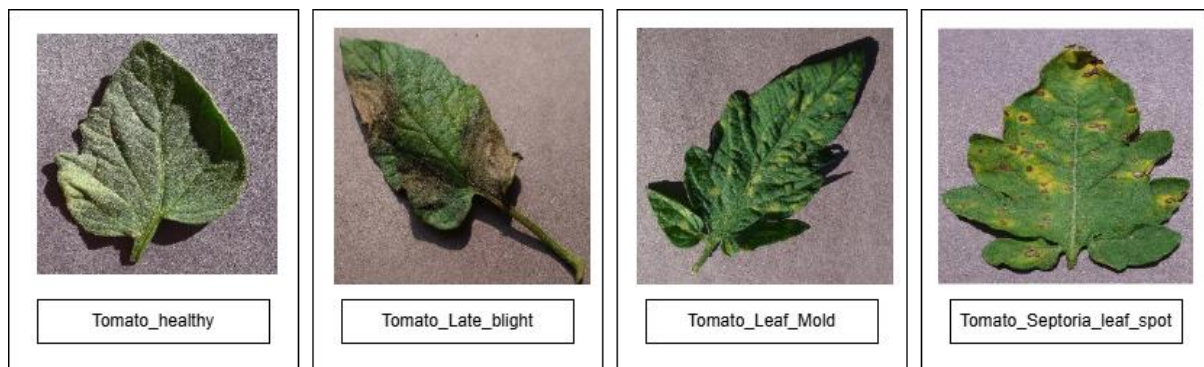
Adathalmaz bemutatása és műveletek

A dokumentáció korábbi fejezeteiben említésre került a KaggleHub által biztosított PlantVillage adathalmaz, melyet a modell tanítási bemenetének használtam. Az adatkészlet különösen alkalmas gépi látási projekt elkészítéséhez, hiszen sok címkézett képet tartalmaz, egységes struktúrával rendelkezik és többféle betegségtípust tartalmaz. A hátránya viszont az, hogy valós mezőgazdasági viszonyok között változóak a fényviszonyok, zajosabb a háttér, részleges takarások lehetnek és előfordulhat rosszabb képminőség is.

A teljes adatbázisból csak egy részhalmaz került feldolgozásra melyek a következők:

- Tomato_healthy
- Tomato_Late_blight
- Tomato_Leaf_Mold
- Tomato_Septorial_lead_spot.

Az egyes csoportok különböző típusú paradicsomlevél betegségeket tartalmaznak. A „healthy” osztályban egészséges levelekről vannak képek, míg a többi mind különböző kórokozó, betegség. A bementi képekből pár szemléltető példa az alábbi **1. ábra** Osztálytípusok képei - án látható.



1. ábra Osztálytípusok képei

A projekt során a dataset mappastruktúra alapú felépítésben lett tárolásra. Az osztályok külön mappákban helyezkednek el, amelyek által a torchvision ImageFolder használata megvalósulhatott. A mappastruktúra a következő volt:

```
plantvillage_small/
├── Tomato_healthy/
├── Tomato_Late_blight/
├── Tomato_Leaf_Mold/
└── Tomato_Septoria_leaf_spot/
```

Ebben a formában minden mappa egy betegség osztályt tartalmazott, amelyeken belül voltak a képek. Az osztálynevek a betöltés alatt automatikusan címkékké alakultak.

Mivel a PlantVillage nagy méretű adatgyűjtemény, a futási sebesség csökkentése érdekében csak 2000 képet használtam egyszerre a tanításhoz. A kiválasztásuk véletlenszerűen történt. A kiválasztott képeket három részre osztottam fel. A tanításra 1400 darab képet, a validációra 300 és a tesztelésre is 300 képet használtam. A train-validation-test struktúra célja, hogy ellenőrizhető a folyamat és az eredmény objektíven mérhető.

Adatelőkészítés

A mélytanulási modellek hatékonyságát nagyban befolyásolja a bementi adatok minősége és az előfeldolgozás, így szükség volt az adatok megfelelő előkészítésére. A cél az volt, hogy a képeknek legyen egységes formátumuk, hogy ilyen formában kerüljenek a neurális hálózat bemenetére. Ezen feldolgozás a torchvision.transforms modul segítségével készült el és az adatfelkészítés két részre bontható, a train transzformációkra és a validáció és test transzformációkra. A train képek esetében augmentációt is alkalmaztam, de a validáció, test

adatok esetében csak előfeldolgozás történt, hogy a modell teljesítménye megfelelően mérhető legyen.

A képekkel a következő átalakítások, módosítások történtek:

- átméretezés
- tensor konverzió
- horizontális tükrözés
- Gaussian blur
- Gaussian noise

A képek átméretezése azért volt fontos, mert a neurális hálózatok fix bemeneti méretet igényelnek. A projektben használt képeket, egységesen 128x128 pixel nagyságúra lettek átalakítva. A képek méreteinek átalakítása, csökkentése számos előnnyel járt. Ezáltal gyorsabb lett a tanítási folyamat, kisebb memóriaigény kellett hozzá, rövidebb lett a futási idő és stabilabb a batch feldolgozás. Adott esetben az átméretezés járhatott volna kisebb részletek elvesztésével, de jelen esetben az említett méret megfelelőnek bizonyult.

A tensor konverzióra azért volt szükség, mert az eredeti letöltött képek RGB képként, képfájl formátumúak, azonban a PyTorch modellek tensor formátumú bemeneti képeket várnak.

Ehhez a `ToTensor()` transzformációt alkalmaztam, amely során pixelek normalizált lebegőpontos értékekké konvertálódnak, azaz 0-255 értékeket 0-1 közé skálázza. Ez fontos volt továbbá a neurális hálózat megfelelő működéséhez és a stabil tanításhoz.

Ezen felül a horizontális tükrözés implementálására is szükség volt, hiszen így csökkenthető volt a túlilleszkedés és javítható volt a modell általánosítható képessége. Ezáltal a modell nem csak egy levélrendezést tanult meg, hanem különböző orientációk mellett is fel tudja ismerni a betegségeket. A tükrözés a `RandomHorizontalFlip` 50%-os valószínűséggel történő alkalmazásával készült.

A blur kísérletek során Gaussian blur transzformációt alkalmaztam a tanító képeken. Ez egy olyan elmosási transzformáció, ami csökkenti a képélességet és kisimítja a részleteket. Ezzel valamilyen fokig utánozhatók a valós környezeti jelenségek, mint az életlen kamera, mozgás közbeni kép készítése, fókuszhibák vagy gyenge optikai minőség. A projektben a blur egy 5x5 méretű kernellel lett megvalósítva, aminek hatására a képen lévő finom textúrák csökkentek, élek életlenebbé váltak, de a jelentősebb mintázatok és színterületek megmaradtak.

A noise kísérletek során Gaussian noise transzformációt alkalmaztam, amelynél minden pixelhez normalizált eloszlást követő érték került random, véletlenszerűen. A valós

környezetből ez a kamerazajt, a rossz fényviszonyokat, az alacsony képminőséget és a szenzorhibákat modellezi. Ez azért lehetséges mert a zaj nagyban megváltoztatja a képen a textúrákat, a foltok szerkezetét és a finom színekülönbségeket.

Modell bemutatása

A modell elkészítéséhez egy konvolúciós neurális hálózat került felhasználásra. A használt eszköz a MobileNetV2 volt, ami egy előre tanított mélytanulási neurális hálózat. A fő feladat az volt, hogy a 4 paradicsomlevél osztályba sikeresen besoroljam a megadott képeket.

A képfeldolgozási feladatoknál alapvetően a hagyományos teljesen összekapcsolt neurális hálózatok kevésbé hatékonyak, hiszen rengeteg pixel található egy képen. Ezt a problémát úgy oldották meg, hogy képesek megtanulni a képeken található kiemelkedő mintázatokat és jellemzőket. A CNN modellek működésének alapja a konvolúciós réteg, ami kis méretű szűrőket alkalmaz a bemeneti képekre, keresi a mintázatokat és fokozatosan egyre összetettebb jellemzőket tanul meg. A korai rétegek jellemzően éleket, színátmeneteket és egyszerű formákat ismerne fel, de a mélyebb rétegek képesek megtanulni összetett textúrákat, betegségre jellemző struktúrákat. A CNN modellek egyik legnagyobb előnye, hogy a releváns kép jellemzőket képesek automatikusan megtanulni, manuális feature engineering nélkül (IBM, 2026).

A feladatban használt MobileNetV2 architektúra egyik kiemelkedő tulajdonsága, hogy úgynevezett depthwise separable convolution használ. Ez egy olyan optimalizált megoldás, ami nagyon lecsökkenti a számítási igényt a hagyományos konvolúciókhoz képest (PyTorch, 2026).

A modellben használok úgynevezett inverted residual block struktúrát, ami lehetővé teszi a hatékonyabb feature extractiont, csökkenti a szükséges paraméterszámot és gyorsabb tanítást eredményez. A MobileNetV2 a projekt szempontjából optimális választásnak bizonyult, hiszen CPU-n is viszonylag gyorsan futott, kisebb adatkészletnél is megfelelően működött és előre tanított súlyok elérhetőek voltak.

A modellt viszont nem teljesen neurális hálóra tanítottam, hanem transfer learning módszerrel, előre betanított neurális hálót használtam. A MobileNetV2 eredetileg az ImageNet adatkészletre lett betanítva, ami több mint egymillió képet és 1000 osztályt tartalmaz. A tanítási folyamat alatt a modell már megtanulta az alapvető képi mintázatokat, textúrákat, éleket és egyéb struktúrákat. Így a korábban megtanult vizuális jellemzők lettek felhasználva a növénybetegségek felismeréséhez. Az említett 1000 ImageNet osztály felismerésére nem volt

szükség, hiszen csak 4 osztályt kellett jelen esetben felismerni. Ezáltal az utolsó osztályozó réteget módosítani kellett. Az eredeti kimeneti réteget egy új teljesen összekapcsolt rétegre cseréltem, ahogy a **2. ábra** Réteg csere is mutatja.

```
def build_model(num_classes, device):  
    model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)  
    model.classifier[1] = nn.Linear(model.last_channel, num_classes)  
    return model.to(device)
```

2. ábra Réteg csere

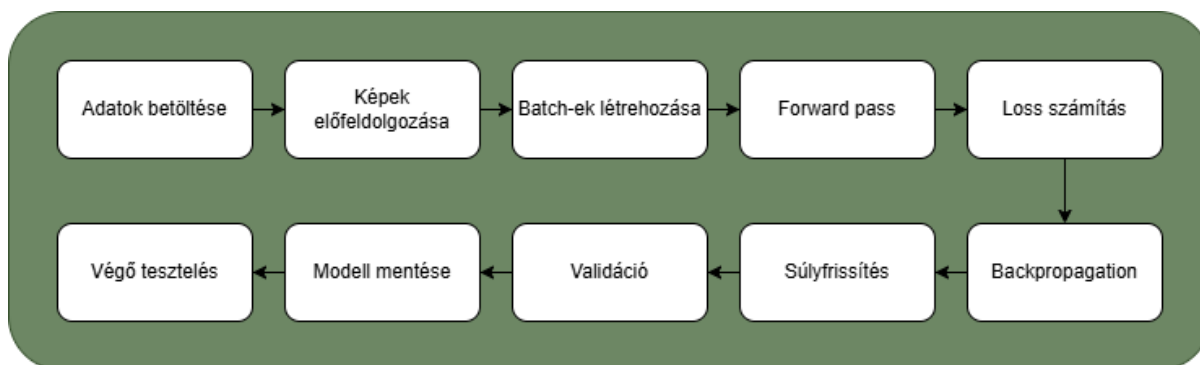
Ebben a kódrészletben a *model.last_channel* a MobileNetV2 utolsó feature vektorának méretét takarja és a *num_classes* pedig a projektben lévő osztályok számát, azaz négyet. Az új réteg feladata az volt, hogy kivont jellemzők alapján osztályozza a képeket és meghatározza a legvalószínűbb betegséget.

A modell bemenete 128x128 pixel méretű RGB képek voltak, amelyek a fenti fejezetekben a megfelelő módon feldolgozásra kerültek. A feldolgozás után a kép áthaladt a konvolúciós rétegen, amely folyamatosan jellemzőket nyert ki, majd a végső feature reprezentáció az osztályozó réteghez került és végül a modell minden osztályhoz egy valószínűségi értéket rendelt. A kimenet a négy osztálytípusból állt, és a legnagyobb értékkel rendelkező lett a modell predikciója.

A modell alapvetően jól teljesített a projekt során. Előnyei, hogy magas baseline pontosságot adott, alacsony számításigény és gyors futás mellett. Ugyanakkor a később bemutatott kísérletekben látható, hogy a modell jelentősen érzékeny bizonyos zavaró tényezőkre.

Tanítási folyamat

A feladat során a neurális hálózat tanítása felügyelt tanulási módszerrel (supervised learning) lett megvalósítva. A bemenetként szolgáló képek előre címkézve voltak, amelyekhez az osztálycímek ismertek voltak. A tanulás teljes folyamatát az **3. ábra** Tanítás folyamata mutatja.



3. ábra Tanítás folyamata

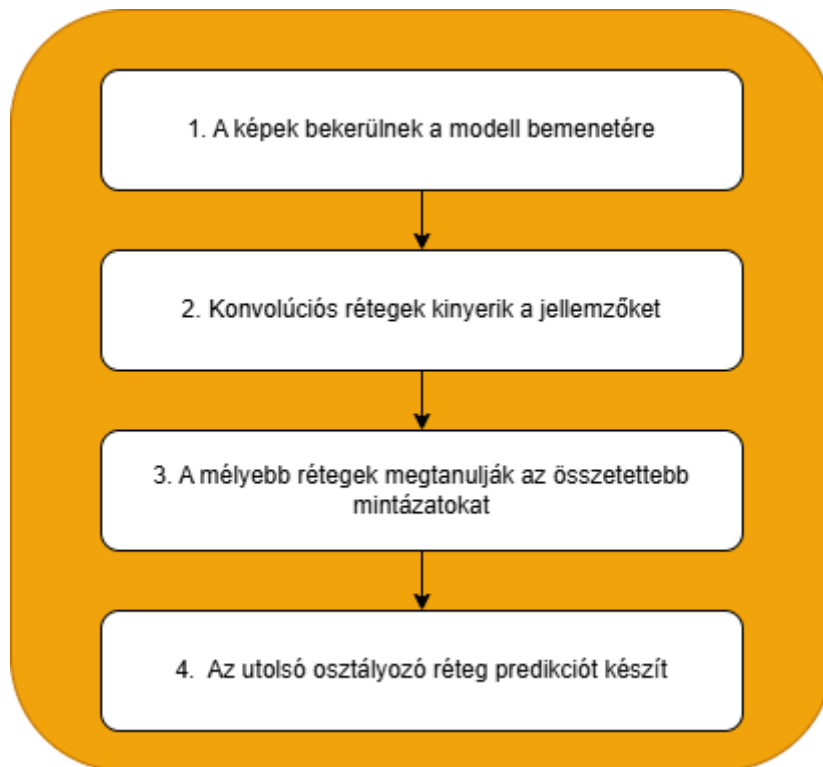
A tanítás legelső lépése az adatok betöltése volt, amelyben a mappákban tárolt képeket a *torchvision.datasets.ImageFolder* osztály segítségével töltött be a program. Az ImageFolder minden mappát külön osztályként kezel és automatikusan hozzárendel címkeket a képekhez, továbbá előkészíti az adatokat a DataLoader számára. Az adatkészlet betöltése során kettő példány készül el, egy a train transzformációkkal és egy az evaluation transzformációkkal. Ez azért történt így, hogy a tanító képhalmazokon augmentációt és zajosítást lehessen alkalmazni, miközben a validációs és teszt képek torzításmentesek maradnak.

A teljes felhasznált adatmennyiség 2000 képből állt össze, amelyek véletlenszerűen kerültek felosztásra a train (1400db), validation (300 db) és test (300db) halmazba.

A train halmazba tartozó képek szolgálták a modell teljes tanítására, azaz ezeken az adatokon módosított súlyokat és tanulta meg a különböző mintázatokat. A validációs halmaz célja az volt, hogy minden epoch végén lehessen ellenőrizni a modell általánosításának mértékét és a túlilleszkedést. Ezen a szakaszon a modell már nem tanult, csak kiértékelés történt. A teszt adathalmaz egyedül a végső mérésre szolgált, hiszen ezeket a képeket a modell a tanulás során nem látta, így objektív teljesítménymérésre voltak alkalmasak.

A képeket batch-ekben dolgoztam fel, és a batch mérete 16 volt, azaz a modell egyszerre 16 képet dolgozott fel. Ennek előnye volt a gyorsabb számítás, a stabilabb gradient becslés, kisebb memórialhasználás és a hatékonyabb CPU/GPU kihasználás. A DataLoader feladata volt a képek batch-ekbe rendezése, továbbá a train adatok megkeverése és a párhuzamos adatbetöltés. A tanító DataLoader esetében a képeket minden epoch előtt véletlenszerűen megkevertem (*shuffle=true*), így a modell nem tudta megtanulni a képek sorrendjét.

A forward pass folyamatban (melyet a **4. ábra** Forward pass mutat) a tanítás során minden batch végig ment a neurális hálózaton.



4. ábra Forward pass

A modell minden képhez hozzáad négy értéket, amely a négy betegséget reprezentálja, és a nagyobb értéket kapó osztály lesz a modell predikciója.

A predikció megállapítása után a rendszer kiszámítja a loss értékeket a `CrossEntropyLoss` függvény segítségével. A loss megmutatja, hogy mennyire tér el a becsült eredmény a valódi címkéktől, azaz mennyire hibás a modell megállapítása. Abban az esetben ha a modell helyes osztályt választ és nagy súlyozással teszi ezt, akkor alacsony a loss értéke. Ellenben, ha rossz vagy bizonytalan akkor magas lesz az értéke. A cél a loss minimalizálása volt.

A következő lépés a backpropagation volt, amelynek a feladata, hogy állapítsa meg, a modell mely súlyai okozták a hibát, a gradiensek kiszámítása és a súlyfrissítés előkészítése. A PyTorch ezt a műveletet automatikusan végzi a `loss.backward()` utasítás meghívásával.

A gradientek kiszámolását követően az Adam optimizert használtam a súlyok frissítésére minden batch után. A cél ezzel az, hogy csökkenjen a loss, javítsa a becsléseket és folyamatosan optimalizálja a modellt.

A tanítás alapvetően epoch alapú rendszerben történt, azaz a modell minden epoch alkalmával végigmegy egyszer az összes képen. Az epochok alatt a modell folyamatosan javította a teljesítményét, ugyanis növekedett a train accuracy, csökkent a loss és stabilizálódott a validation accuracy is.

Minden epoch befejeztével történt egy validációs folyamat, amely során a modell kiértékelési módba került (*model.eval()*), nem történt súlyfrissítés, csak teljesítménymérés történt. Így ellenőrizte a program minden epoch után, hogy jobb lett-e az accuracy, és ha igen, automatikusan mentésre került a modell. A mentés a *torch.save(model.state_dict(), best_path)* használatával történt, ami biztosította, hogy a végső teszteléshez a legjobban teljesítő modell kerüljön felhasználásra.

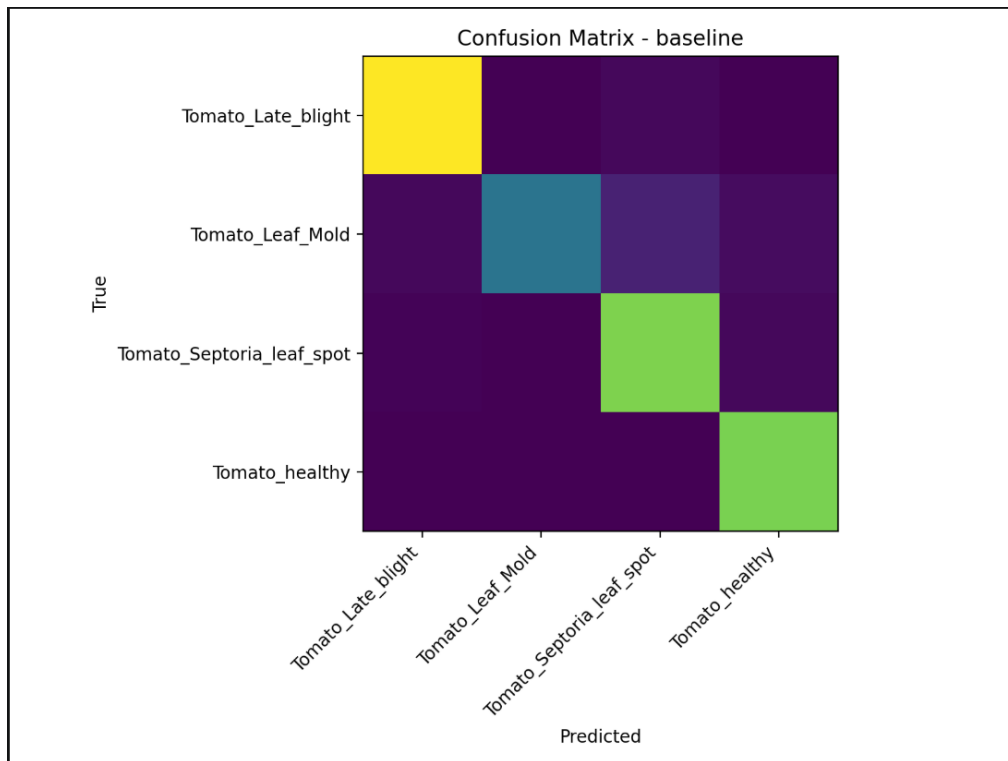
A tanítás befejezése után a program betöltötte a legjobban teljesítő modellt és lefuttatta a teszt halmazon. Eközben nem volt tanulás csak kiértékelés. A modell teljesítményének mérése érdekében az accuracy, precision, recall, F1-score és confusion matrix metrikák kerültek felhasználásra. Ez lehetővé tette, hogy lehessen mérni a teljesítményt és elemezni az osztályszintű hibákat.

Kísérletek és eredmények

A projekt során három különböző kísérletet végeztem a modell működésének vizsgálatához. Minden kör alatt ugyanaz az adatkészlet (PlantVillage) került felhasználásra, ugyanazok voltak a tanítási paraméterek és a MobileNetV2 architektúra működött közben. A különbség minden kísérletben kizárólag a képi transzformációk voltak. A három eset a baseline, Gaussian blur és Gaussian noise volt. Ezek által lehetőség volt annak vizsgálatára, hogy a melyik torzítások rontják leginkább a modellt, mennyire robosztus a modell és, hogy milyen típusú képi transzformációkra támaszkodik a neurális hálózat.

Baseline kísérlet

A baseline kísérlet futtatásának az volt a célja, hogy meghatározzam a rendszer alap teljesítményét optimális körülmények között. A tanító képeken kizárólag alap augmentáció történt, mint a képátméretezés, a horizontális tükrözés és a tensor konverzió. A baseline futás eredményeit a **5. ábra** Baseline report tartalmazza.



6. ábra Baseline confusion matrix

A confusion matrix alapján megállapítható, hogy az osztályok jól elkülönültek, a modell stabil teljesítményt nyújtott és a transfer learning jól működött.

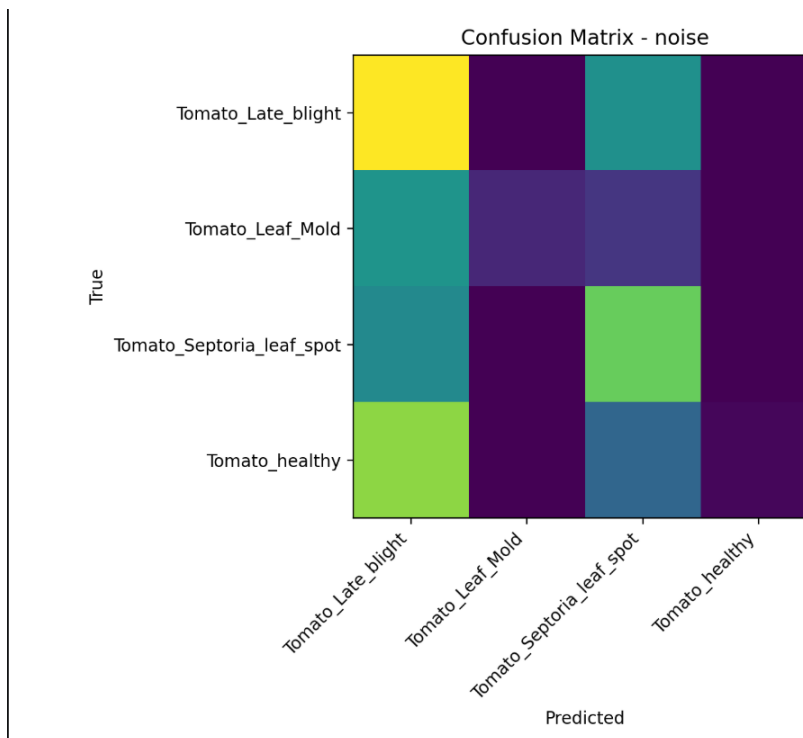
Gaussian noise kísérlet

A második kísérlet során Gaussian noise transzformáció került alkalmazásra a tanító képeken. A zaj minden egyes pixelhez véletlenszerű eltérést csatolt hozzá normális eloszlás alapján. A zajosításnak az volt a célja, hogy modellezsem a valós környezeti problémákat. A kísérlet eredményeit a **7. ábra** Noise report mutatja.

Experiment: noise														
Test accuracy: 0.4033														
Test loss: 2.1964														
										precision	recall	f1-score	support	
										Tomato_Late_blight	0.36	0.67	0.46	96
										Tomato_Leaf_Mold	1.00	0.14	0.25	50
										Tomato_Septoria_leaf_spot	0.44	0.62	0.51	79
										Tomato_healthy	1.00	0.01	0.03	75
										accuracy			0.40	300
										macro avg	0.70	0.36	0.31	300
										weighted avg	0.65	0.40	0.33	300

7. ábra Noise report

A Gaussian noise jelentős teljesítmény csökkenést okozott, hiszen az accuracy 94%-ról lecsökkent 40%-ra. A report alapján több osztály precision és recall értéke leromlott és a modell bizonytalanabbá vált. Különösen problémás volt a Tomato_healthy és a Tomato_Leaf_Mold osztály felismerése. A Tomato_healthy recall értékének alacsonysága azt jelenti, hogy a modell szinte egyáltalán nem tudta felismerni az egészséges leveleket zajos környezetben. Az 8. ábra Noise confusion matrix mutatja a noise kísérlethez tartozó confusion matrix képét, amelyen látható, hogy az átló kevésbé domináns, az osztályok nagyon összekeveredtek és nőtt a félreosztás száma jelentősen.



8. ábra Noise confusion matrix

A modell elveszítette a stabil felismerési képességét, a zaj túlzottan torzított a fontos képi jellemzőket, így az osztályok közötti különbségtétel nehezebbé vált. Ezen alapján megállapítható, hogy a neurális hálózat különösen támaszkodik a textúrákra, finom részletekre és a színmintázatokra.

Gaussian blur kísérlet

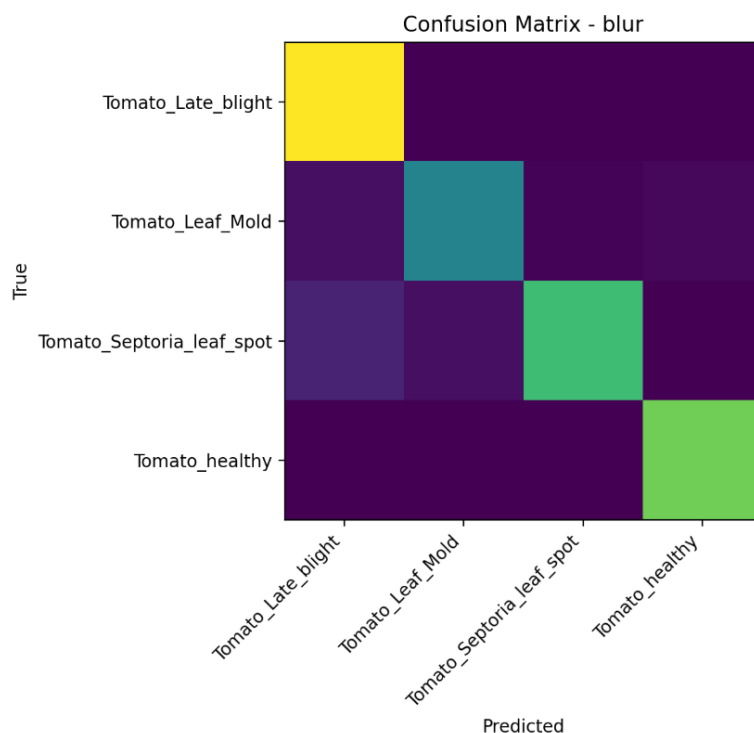
A harmadik kísérlet során Gaussian blur került alkalmazásra a bemeneti képeken, ami egy olyan elmosási művelet, ahol csökken a képek élessége és kisimulnak a részletek. A Gaussian blur kevésbé torzítja a teljes kép struktúráját, mint a zajosítás, de részben eltávolítja a finom részleteket. A blur eredményei az alábbi **9. ábra** Blur report report mutatja.

```
Experiment: blur
Test accuracy: 0.9333
Test loss: 0.1985
```

											precision	recall	f1-score	support	
											Tomato_Late_blight	0.88	1.00	0.94	96
											Tomato_Leaf_Mold	0.91	0.86	0.89	50
											Tomato_Septoria_leaf_spot	0.99	0.84	0.90	79
											Tomato_healthy	0.97	1.00	0.99	75
											accuracy			0.93	300
											macro avg	0.94	0.92	0.93	300
											weighted avg	0.94	0.93	0.93	300

9. ábra Blur report

Az eredmények alapján megállapítható, hogy a blur minimális teljesítménycsökkenést okozott a baseline kísérlethez képest. A legtöbb osztály továbbra is magas precision és recall értéket ért el, a modell stabil maradt és kevés volt a hibás osztályozás. A confusion matrix (10. ábra Blur confusion matrix) jól mutatja, hogy a főbb mintázatok továbbra is felismerhetőek maradtak, megfelelően elkülönültek az osztályok és a modell aránylag jól tolerálta az enyhe életlenséget.



10. ábra Blur confusion matrix

Tehát kijelenthető, hogy a neurális hálózat nem kizárólag apró részletekre támaszkodik, a nagyobb levélmintázatk is fontos szerepet játszanak a felismerés folyamatában.

Irodalomjegyzék

- Cournapeau, D. (2026). *user guide*. Forrás: scikit learn: https://scikit-learn.org/stable/user_guide.html
- Foundation, P. S. (2026). *Python 3.14 documentation*. Forrás: Python: <https://docs.python.org/3/>
- Hunter, J., & Dale, D. (2026). *Using Matplotlib*. Forrás: matplotlib: <https://matplotlib.org/stable/users/index>
- IBM. (2026). *What are convolutional neural networks?* Forrás: imb.hu: <https://www.ibm.com/think/topics/convolutional-neural-networks>
- Kaggle. (2018). *PlantVillage Kaggle*. Forrás: Kaggle: <https://www.kaggle.com/datasets/emmarex/plantdisease>
- PyTorch. (2026). *PyTorch Documentation*. Forrás: PyTorch: <https://docs.pytorch.org/docs/2.12/index.html>

Ábrajegyzék

1. ábra	Osztálytípusok képei.....	6
2. ábra	Réteg csere.....	9
3. ábra	Tanítás folyamata.....	10
4. ábra	Forward pass.....	11
5. ábra	Baseline report.....	13
6. ábra	Baseline confusion matrix.....	14
7. ábra	Noise report.....	15
8. ábra	Noise confusion matrix.....	16
9. ábra	Blur report.....	17
10. ábra	Blur confusion matrix.....	17